

Лабораторная работа 4: "Численное решение обыкновенных дифференциальных уравнений"

Задание

1. Дана задача Коши для ОДУ. Решить эту задачу на данном интервале $[x_0, X]$ аналитически или численно (самостоятельно или с помощью Mathematica или любой другой программы). Разрешается вместо задачи из варианта взять любую задачу небесной или земной механики.
2. Построить график полученного решения, вычислить значение решения в точке X . Если дана система ОДУ, необходимо построить графики каждой компоненты решения.
3. Написать программу, которая решает данную задачу Коши соответствующим вашему варианту численным методом с адаптивным выбором шага по правилу Рунге (см. конспект). Если метод многошаговый - используйте фиксированный шаг, стартовые значения следует брать из решения, полученного в пункте 1. Если ОДУ имеет порядок выше первого, его необходимо предварительно свести к системе первого порядка путем введения вспомогательных переменных.
4. Построить точечные графики численного решения, полученного количестве шагов около $N = 200$, совместить их с графиками из пункта 2.
5. Построить логарифмическую диаграмму сходимости. Для многошаговых методов с постоянным шагом по оси абсцисс на диаграмме откладывается величина шага h , соответствующая количеству отрезков $N = 10^k$, $k = 1, 2, \dots, 6$. Для одношаговых методов с адаптивной сеткой по оси абсцисс откладывается требуемая величина локальной погрешности $\varepsilon = 10^{-k}$, $k = 1, 2, \dots, 10$. По оси ординат откладывается погрешность $r_N = |y(X) - y_N|$. Здесь $y(X)$ - точное решение, вычисленное в пункте 2, y_N - приближенное решение, полученное вашей программой.
6. **Составить отчет**, который содержит
 - Описание решения задачи Коши из пункта 1. Если используются сторонние программы - привести команды, с помощью которых получено решение;
 - Сведение к системе первого порядка (если решается уравнение порядка выше 1);
 - Совмещенные графики точного и приближенного решений из пункта 4;
 - Диаграмма из пункта 5;
 - Код программы.

Метод

Двухшаговый явный метод Адамса.

Задача

$$\begin{cases} u'(x) = 3u(x) - 2u(x)v(x) \\ v'(x) = u(x)v(x) - 2v(x) \end{cases}$$

$$u(0) = 2, v(0) = 1, [x_0, X] = [0, 10]$$

Решение

1, 2

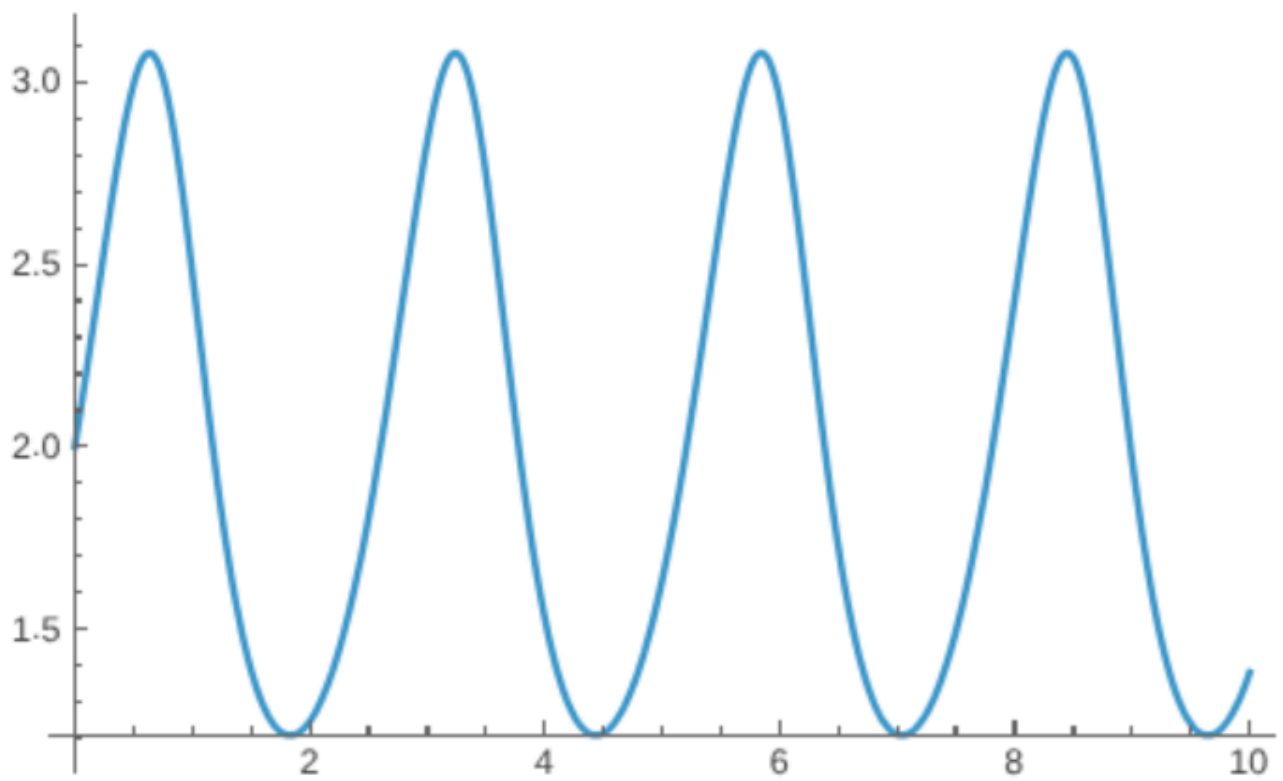
Решим поставленную задачу численно при помощи Wolfram Mathematica. Код:

```
sol=NDSolve[
{
u'[x]\[Equal]3u[x]-2u[x]v[x],
v'[x]\[Equal]u[x]v[x]-2v[x],
u[0]\[Equal]2,
v[0]\[Equal]1},
{u,v},
{x,0,10},
WorkingPrecision->20,
AccuracyGoal -> 15,
PrecisionGoal->15
][[1]]
{u[10],v[10]}/.sol

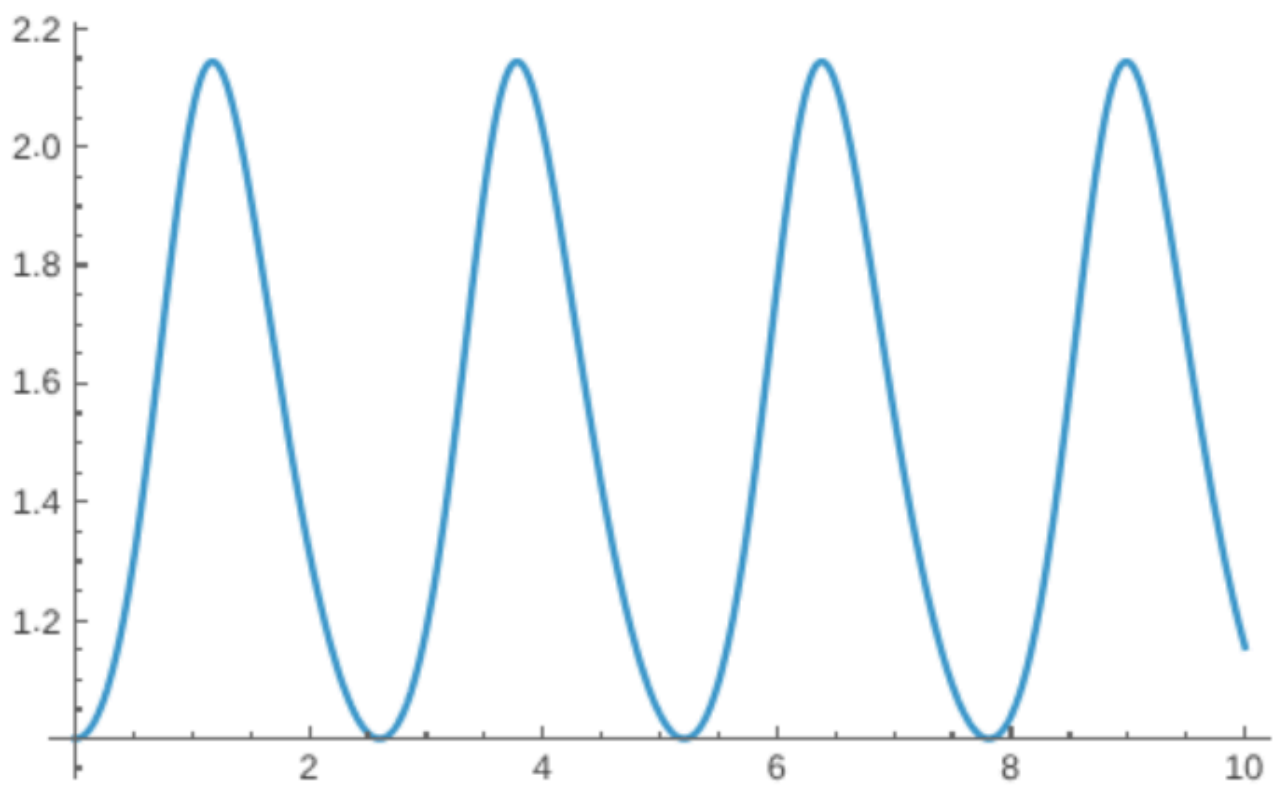
{a,b}={0,10};
h = (b-a) / 200;
NumberForm[{u[h],v[h]}/.sol, 14]

Plot[u[x]/.sol,{x,a,b}]
Plot[v[x]/.sol,{x,a,b}]
```

Получили графики функций:



$u(x)$



$v(x)$

и решение в точке $X = 10$:

```
{u[10], v[10]} = {1.379677718794615029, 1.1549517595510732994}
```

$$y(10) = \begin{bmatrix} u(10) \\ v(10) \end{bmatrix} = \begin{bmatrix} 1.379677718794615029 \\ 1.1549517595510732994 \end{bmatrix}$$

3

Двухшаговый метод Адамса выглядит следующим образом

$$y_2 = y_1 + h \left(\frac{3}{2} f_1 - \frac{1}{2} f_0 \right)$$

Для нашей задачи:

$$f_i = \begin{bmatrix} 3u(x_i) - 2u(x_i)v(x_i) \\ u(x_i)v(x_i) - 2v(x_i) \end{bmatrix}$$

Начальные точки (y_1 взято из решения в Mathematica):

$$x_0 = 0, y_0 = \begin{bmatrix} 2 \\ 1 \end{bmatrix}, y_1 = \begin{bmatrix} 2.10236474860118 \\ 1.00254324840088 \end{bmatrix}$$

```
#include <array>
#include <fstream>
#include <iomanip>
#include <string>
#include <vector>

static constexpr size_t kDim = 2;

using Vec = std::array<double, kDim>;

constexpr Vec operator+(Vec a, Vec b) {
    Vec res;
    for (size_t i = 0; i < kDim; ++i) {
        res[i] = a[i] + b[i];
    }
    return res;
}

constexpr Vec operator-(Vec a, Vec b) {
    Vec res;
    for (size_t i = 0; i < kDim; ++i) {
        res[i] = a[i] - b[i];
    }
    return res;
}

constexpr Vec operator*(double a, Vec b) {
```

```

    Vec res;
    for (size_t i = 0; i < kDim; ++i) {
        res[i] = a * b[i];
    }
    return res;
}

static constexpr double kA = 0.;
static constexpr double kB = 10.;
static constexpr Vec kY0 = {2., 1.};

// change
static constexpr size_t kN = 200;
static constexpr Vec kY1 = {2.10236474860118, 1.00254324840088};

static constexpr double kH = (kB - kA) / kN;
std::string kPath = "output" + (kN == 200 ? "" : std::to_string(kN)) +
".csv";

// specific task
constexpr Vec F(Vec y) {
    Vec res{0};
    auto [u, v] = y;
    res[0] = 3 * u - 2 * u * v;
    res[1] = u * v - 2 * v;
    return res;
}

// Adams method
constexpr Vec Yi(Vec yi_1, Vec fi_1, Vec fi_2) {
    return yi_1 + kH * (1.5 * fi_1 - 0.5 * fi_2);
}

int main() {
    std::ofstream fout(kPath);

    std::vector<std::array<double, kDim>> ys = {kY0, kY1};
    ys.resize(kN + 1);

    Vec f_prev = F(kY0);
    for (size_t i = 2; i < ys.size(); ++i) {
        Vec f_curr = F(ys[i - 1]);
        ys[i] = Yi(ys[i - 1], f_curr, f_prev);
        f_prev = f_curr;
    }
}

```

```

double x = kA;
fout << std::setprecision(14);
for (auto [u, v] : ys) {
    fout << x << "," << u << "," << v << "\n";
    x += kH;
}

return 0;
}

```

Получили $y(10) = \begin{bmatrix} 1.4343986899592 \\ 1.1127194693485 \end{bmatrix}$, в файле `data.csv` получаем точки для графика.

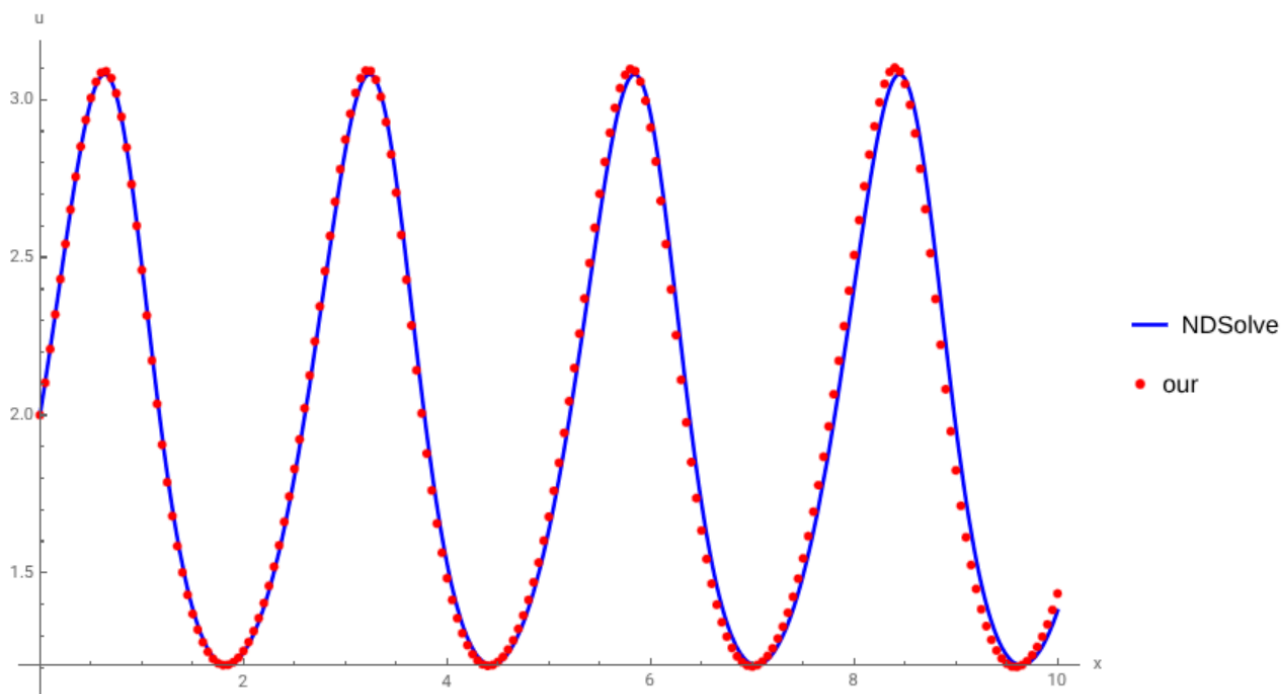
4

при помощи Mathematica построим совмещенные графики точного и нашего решения, код следующий:

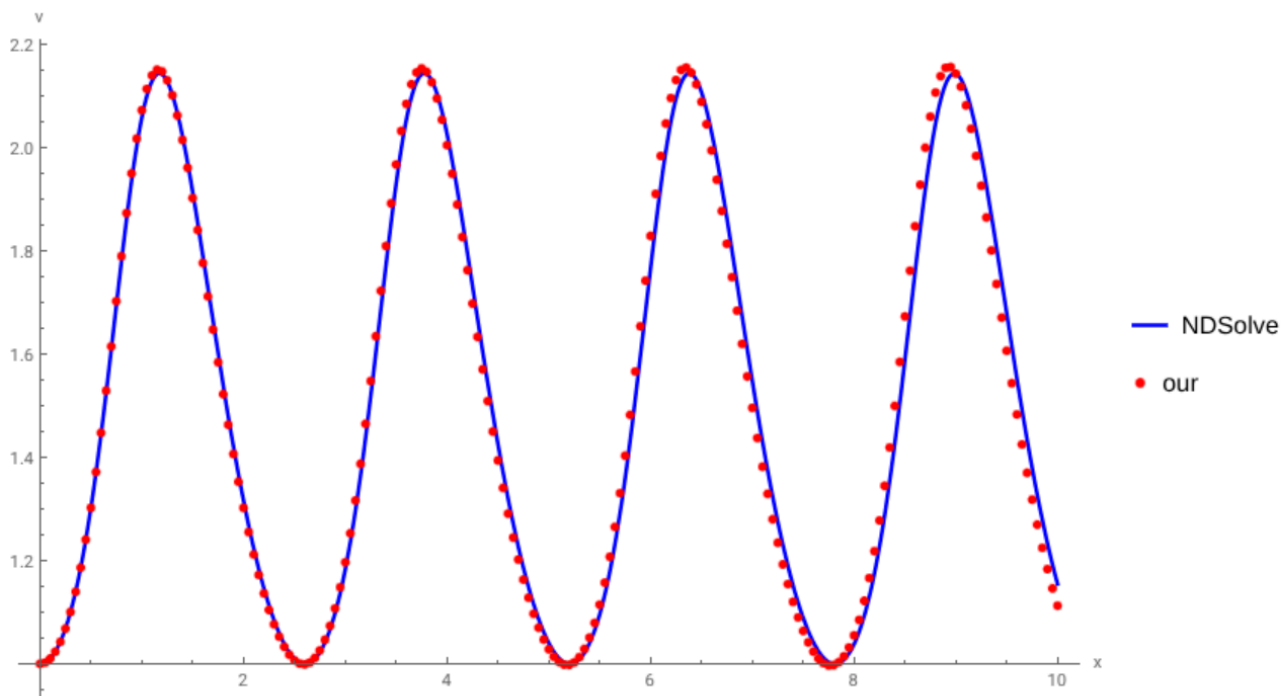
```

SetDirectory[NotebookDirectory[]];
data = Import["output.csv", "CSV"];
xDat = data[[All, 1]];
uDat = data[[All, 2]];
vDat = data[[All, 3]];
uPts = Transpose[{xDat, uDat}];
vPts = Transpose[{xDat, vDat}];
Show[
    Plot[u[x] /. sol, {x, 0, 10}, PlotStyle -> Blue, PlotLegends ->
{"NDSolve"}, ImageSize->Large],
    ListPlot[uPts, PlotStyle -> {Red, PointSize[0.008]}, PlotLegends ->
{"our"}],
    AxesLabel -> {"x", "u"}
]
Show[
    Plot[v[x] /. sol, {x, 0, 10}, PlotStyle -> Blue, PlotLegends ->
{"NDSolve"}, ImageSize->Large],
    ListPlot[vPts, PlotStyle -> {Red, PointSize[0.008]}, PlotLegends ->
{"our"}],
    AxesLabel -> {"x", "v"}
]

```



$u(x)$



$v(x)$

5

Для получения решений запустим программу, поменяв количество отрезков kN и y_1 . Код для получения y_1 из точного решения (получаем для разных n):

```

n = 100;
h = (b-a)/ n;
y1 = NumberForm[{u[0+h],v[0+h]}/.sol, 14];
Print[y1]

```

Код для получения диаграммы в Mathematica:

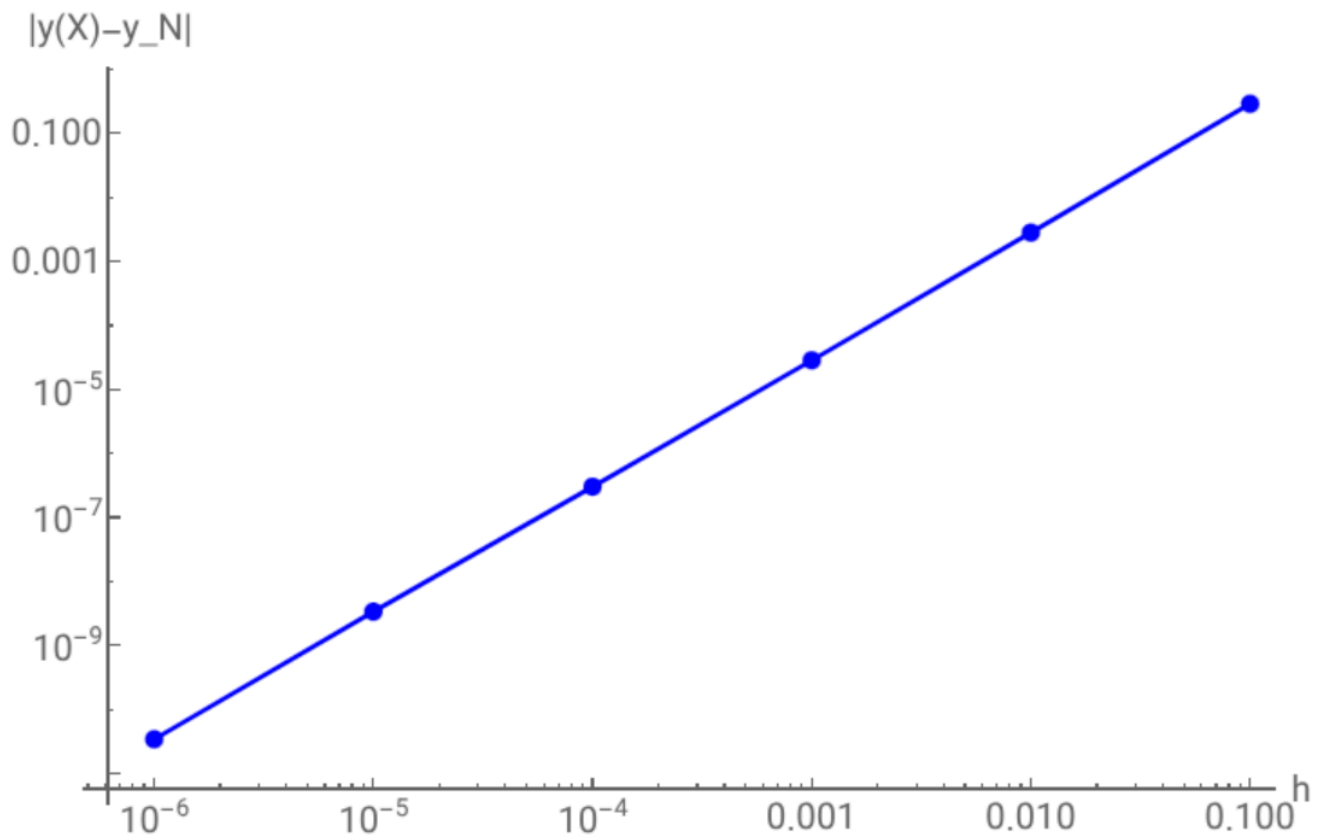
```

final={{1.6245778236896,1.0004524334911},
        {1.381917244918,1.1532419132679},
        {1.3797005978064,1.1549347671222},
        {1.3796779685884,1.1549515896002},
        {1.3796777217119,1.1549517578482},
        {1.3796777188243,1.1549517595339}};
acc={u[10],v[10]}/.sol;
nVals={10^2, 10^3,10^4, 10^5,10^6, 10^7};
hVals=(b-a)/nVals;
errs=Norm[#-acc] &/@final;

ListLogLogPlot[
  Transpose[{hVals, errs}],
  PlotStyle->{Red, PointSize[0.02]},
  Joined->True,
  Mesh->All,
  AxesLabel->{"h", "|y(10)-y_N|"},
  PlotRange->Automatic,
  ImageSize->Large
]

```

Получили:



Для $N = 10$ решение расходится, поэтому не учитываем. Видим, что погрешность уменьшается с уменьшением отрезка шага.